

[CSED211] Introduction to Computer Software Systems

Lab 3: Bomb Lab

Hyunbin Pak



CAOS
COMPUTER ARCHITECTURE &
OPERATING SYSTEMS LABORATORY

2025.09.25

Today's Agenda

- GCC
- GDB
- Assembly Language
- Memory Layout
- Bomb Lab
- Quiz

- GNU compiler collection
 - C, C++, Go, ...
 - Standard libraries (e.g., `libstdc++`)
- How to compile using GCC?
 - Create an **executable file** (**execfile**) from **.C** (**source_code.c**)
 - `$ gcc -o [execfile] [source_code.c]`
 - Create an **executable file** (**execfile**) from **.S** (**source_code.s**)
 - `$ gcc -o [execfile] [source_code.s]`
 - Create an **assembly file** (**asm.s**) from **.C** (**source_code.c**)
 - `$ gcc -S [source_code.c] -o [asm.s]`

Today's Agenda

- GCC
- **GDB**
- Assembly Language
- Memory Layout
- Bomb Lab
- Quiz

GDB

- GNU project debugger
 - Allows you to see what is going on inside a target program during its execution
- How to execute the target program over GDB?
 - Run the target **program (execfile)** over GDB
 - `$ gdb [execfile]`

```
[sho0927@programming2 ppt]$ ./test
Hello World!
[sho0927@programming2 ppt]$ gdb test
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-120.el7
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from /home/std/sho0927/bomblab_ta_example/ppt/test...(no debug
(gdb) █
```

GDB Commands

- **disassemble (disas)**: Disassemble a specific function by name (e.g., `main`)
 - `(gdb) disas main`
- **run (r)**: Run the program
 - `(gdb) r`
 - Kill the current running program and re-run the program (e.g., `restart`)

```
(gdb) disas main
Dump of assembler code for function main:
   0x000000000040052d <+0>:    push    %rbp
   0x000000000040052e <+1>:    mov     %rsp,%rbp
   0x0000000000400531 <+4>:    mov     $0x4005e0,%edi
   0x0000000000400536 <+9>:    callq   0x400410 <puts@plt>
   0x000000000040053b <+14>:   mov     $0x0,%eax
   0x0000000000400540 <+19>:   pop     %rbp
   0x0000000000400541 <+20>:   retq
End of assembler dump.
(gdb) █
```

GDB Commands

- **break (b):** Set a breakpoint at a specific memory address
 - (gdb) b *main # The function name is pointer to memory address
 - (gdb) b *main+1
 - (gdb) b *0x400536
 - If you have set any breakpoints, GDB will stop at the first breakpoint
- **continue (c):** Continue the program
 - (gdb) c
 - Resume program execution from the current point to the next breakpoint or endpoint
- **si [step instruction], ni [next instruction]:** Execute next instruction
 - (gdb) si # Step into the procedure call
 - (gdb) ni # Next after finishing procedure call

GDB Commands

- **info (i):** Print various debugging information
 - `(gdb) i [option]`
 - `[Command option]`
 - `Registers (r), breakpoints (b) ...`
- **clear (c1):** Clear a breakpoint specified by given memory address
 - `(gdb) c1 *main`
- **delete (d):** Delete a breakpoint specified by given number (all if number is not given)
 - `(gdb) d 2`
 - `(gdb) d # Delete all breakpoints`
- **kill (k):** Kill the current running program
 - `(gdb) k`

GDB Commands

- `print (p)`: Print current **register value**
 - `(gdb) p $rax`
- `x`: Examine **contents of memory**
 - `(gdb) x/[option] [addr]`
 - [Command option]
 - What granularity do you want to display memory values: [**b**(1) , **h**(2) , **w**(4) , **g**(8)]
 - What format should the contents be displayed in: [**t**₍₂₎ , **o**₍₈₎ , **u**₍₁₀₎ , **x**₍₁₆₎]
 - Show the values in **ASCII code format** until a NULL value is encountered: [**s**]

- Examples:
 - `x/gx 0x400`
 - `x/2wt 0x400`
 - `x/s 0x400`

Address	0x400	0x401	0x402	0x403	0x404	0x405	0x406	0x407	0x408
Value	0x41	0x42	0x43	0x44	0x45	0x46	0x47	0x48	0x00

```
$ (gdb) x/s 0x400 "ABCDEFGH"
```

Today's Agenda

- GCC
- GDB
- **Assembly Language**
- Memory Layout
- Bomb Lab
- Quiz

Assembly Language: Opcode

- Assembly language is what a machine really sees and runs
 - Actually, the machine executes binary code (called machine code)
- Syntax
 - **opcode** (what an instruction does) + **operand** (data)

	opcode	operand
0x000000000000014cd <+4>:	push	%rbx
0x000000000000014ce <+5>:	cmp	\$0x1,%edi
0x000000000000014d1 <+8>:	je	0x15cf <main+262>
0x000000000000014d7 <+14>:	mov	%rsi,%rbx
0x000000000000014da <+17>:	cmp	\$0x2,%edi
0x000000000000014dd <+20>:	jne	0x1604 <main+315>
0x000000000000014e3 <+26>:	mov	0x8(%rsi),%rdi
0x000000000000014e7 <+30>:	lea	0x1f8e(%rip),%rsi
0x000000000000014ee <+37>:	call	0x1350 <fopen@plt>

Assembly Language: Opcode

- opcode specifies the **operation** performed by a machine
 - Unary operator (format: `opcode operand`)
 - `pop, inc, dec, jmp, ...`
 - Binary operator (format: `opcode operand1, operand2`)
 - `mov, add, sub, cmp, ...`

Assembly Language: Operand

- operand specifies the **data** used by the operation (opcode)
 - Immediate: Real values
 - `$2, $6, $-1, ...`
 - Register: Register values
 - `%rax, %rsp, %rsi, ...`
 - Memory: Values at specific memory location
 - `$-8(%ebx), $12(%ebx), ...`

Assembly Language: Registers

- x86_64 instruction set architecture (ISA) includes 16 general purpose registers
 - `rbp` and `rsp` are used to manage **stack frame**

<code>%rax</code>	<code>%r8</code>
<code>%rbx</code>	<code>%r9</code>
<code>%rcx</code>	<code>%r10</code>
<code>%rdx</code>	<code>%r11</code>
<code>%rsi</code>	<code>%r12</code>
<code>%rdi</code>	<code>%r13</code>
<code>%rbp</code>	<code>%r14</code>
<code>%rsp</code>	<code>%r15</code>

Assembly Language: Registers

- x86_64 uses **different registers** to access bits
- You can google them or use ChatGPT for more details

64-bit	<u>RAX</u>							
32-bit					<u>EAX</u>			
16-bit							<u>AX</u>	
8-bit							AH	AL
Value (hex)	01	23	45	67	89	AB	CD	EF

Assembly Language: Instructions

- Data movement instruction
 - `mov [src_operand], [dest_operand]`

```
0x000000000000014cd <+4>:    push    %rbx
0x000000000000014ce <+5>:    cmp     $0x1,%edi
0x000000000000014d1 <+8>:    je      0x15cf <main+262>
0x000000000000014d7 <+14>:   mov     %rsi,%rbx
0x000000000000014da <+17>:   cmp     $0x2,%edi
0x000000000000014dd <+20>:   jne     0x1604 <main+315>
0x000000000000014e3 <+26>:   mov     0x8(%rsi),%rdi
0x000000000000014e7 <+30>:   lea     0x1f8e(%rip),%rsi
0x000000000000014ee <+37>:   call    0x1350 <fopen@plt>
```


Assembly Language: Instructions

- Arithmetic instructions
 - `add/sub/mul` [operand1], [operand2]

```
#include <stdio.h>

int main(void) {
    int a = 0;
    a = a + 1;
    a = a - 2;

    int b = 3;
    a = a * b;
    return 0;
}
```

```
0x00000000000001129 <+0>:    endbr64
0x0000000000000112d <+4>:    push    %rbp
0x0000000000000112e <+5>:    mov     %rsp,%rbp
0x00000000000001131 <+8>:    movl    $0x0,-0x8(%rbp)
0x00000000000001138 <+15>:   addl    $0x1,-0x8(%rbp)
0x0000000000000113c <+19>:   subl    $0x2,-0x8(%rbp)
0x00000000000001140 <+23>:   movl    $0x3,-0x4(%rbp)
0x00000000000001147 <+30>:   mov     -0x8(%rbp),%eax
0x0000000000000114a <+33>:   imul    -0x4(%rbp),%eax
0x0000000000000114e <+37>:   mov     %eax,-0x8(%rbp)
0x00000000000001151 <+40>:   mov     $0x0,%eax
0x00000000000001156 <+45>:   pop     %rbp
0x00000000000001157 <+46>:   ret
```

Assembly Language: Instructions

- Compare instruction (set status flags depending on the result)
 - `cmp [operand1], [operand2]`
 - `R = operand2 - operand1`
- Status flags
 - Carry flag (CF)
 - Set if (unsigned) `R < (unsigned) operand1`
 - Zero flag (ZF)
 - Set if `R == 0`
 - Sign flag (SF)
 - Set if `R < 0`
 - Overflow flag (OF)
 - Set if `(operand1 < 0 == operand2 < 0) && (R < 0 != operand1 < 0)`

Assembly Language: Instructions

- Jump instructions
 - Can be used to implement **conditional branch**

Instruction	Jump Condition	Description
jmp Label	1	Direct jump
jmp *operand	1	Indirect jump
je Label	ZF	Equal/zero
jne Label	~ZF	Not equal/not zero
jg Label	~(SF^OF)&~ZF	Greater (>)
jge Label	~(SF^OF)	Greater or equal (>=)
jl Label	SF^OF	Less (<)
jle Label	(SF^OF) ZF	Less or equal (<=)

```
#include <stdio.h>
```

```
int main(void) {  
    int i = 0, sum = 0;
```

```
    for (i = 0; i < 5; ++i) {  
        sum += i;  
    }
```

```
    printf("%d\n", sum);  
    return 0;  
}
```

```
0x0000000000401126 <+0>:  push    %rbp  
0x0000000000401127 <+1>:  mov     %rsp,%rbp  
0x000000000040112a <+4>:  sub     $0x10,%rsp  
0x000000000040112e <+8>:  movl    $0x0,-0x4(%rbp)  
0x0000000000401135 <+15>: movl    $0x0,-0x8(%rbp)  
0x000000000040113c <+22>: movl    $0x0,-0x4(%rbp)  
0x0000000000401143 <+29>: jmp     0x40114f <main+41>  
0x0000000000401145 <+31>: mov     -0x4(%rbp),%eax  
0x0000000000401148 <+34>: add     %eax,-0x8(%rbp)  
0x000000000040114b <+37>: addl    $0x1,-0x4(%rbp)  
0x000000000040114f <+41>: cmpl    $0x4,-0x4(%rbp)  
0x0000000000401153 <+45>: jle     0x401145 <main+31>  
0x0000000000401155 <+47>: mov     -0x8(%rbp),%eax  
0x0000000000401158 <+50>: mov     %eax,%esi  
0x000000000040115a <+52>: mov     $0x402004,%edi  
0x000000000040115f <+57>: mov     $0x0,%eax  
0x0000000000401164 <+62>: call    0x401030 <printf@plt>  
0x0000000000401169 <+67>: mov     $0x0,%eax  
0x000000000040116e <+72>: leave  
0x000000000040116f <+73>: ret
```

Assembly Language: Instructions

- Other instructions
 - `lea` (Load Effective Address)
 - `lea 7(%rdx), %rdi => %rdi = 7 + %rdx`
 - Note that, `mov` stores **value**, while `lea` stores **address** (e.g., pointer)

Assembly Language: Calling Convention

- Calling convention answers
 - How does function **receive parameters** from the caller?
 - How does function **return a value**?

Assembly Language: Calling Convention

- How does function **receive parameters**?
 - Some registers are used to **pass parameters**

Register	Purpose	Saved Across Calls
%rax	Temp register; Return value	No
%rbx	Callee-saved	Yes
%rcx	Used to pass 4th argument	No
%rdx	Used to pass 3rd argument	No
%rsp	Stack pointer	Yes
%rbp	Callee-saved; Base pointer	Yes
%rsi	Used to pass 2nd argument	No
%rdi	Used to pass 1st argument	No
%r8	Used to pass 5th argument	No
%r9	Used to pass 6th argument	No
%r10-r11	Temporary	No
%r12-r15	Callee-saved registers	Yes

```
int test(int a, int b, int c, int d) {  
    printf("%d %d %d %d\n", a, b, c, d);  
    return 3;  
}  
  
int main(void) {  
    int a = 0;  
    int b = 1;  
    int c = 2;  
    int d = 3;  
  
    test(a, b, c, d);  
    return 0;  
}
```

```
0x0000000000401161 <+0>:  push    %rbp  
0x0000000000401162 <+1>:  mov     %rsp,%rbp  
0x0000000000401165 <+4>:  sub     $0x10,%rsp  
0x0000000000401169 <+8>:  movl    $0x0,-0x4(%rbp)  
0x0000000000401170 <+15>: movl    $0x1,-0x8(%rbp)  
0x0000000000401177 <+22>: movl    $0x2,-0xc(%rbp)  
0x000000000040117c <+29>: movl    $0x3,-0x10(%rbp)  
0x0000000000401185 <+36>:  mov     -0x10(%rbp),%ecx  
0x0000000000401188 <+39>:  mov     -0xc(%rbp),%edx  
0x000000000040118b <+42>:  mov     -0x8(%rbp),%esi  
0x000000000040118e <+45>:  mov     -0x4(%rbp),%eax  
0x0000000000401191 <+48>:  mov     %eax,%edi  
0x0000000000401193 <+50>:  call    0x401126 <test>  
0x0000000000401198 <+55>:  mov     $0x0,%eax  
0x000000000040119d <+60>:  leave  
0x000000000040119e <+61>:  ret
```

Assembly Language: Calling Convention

- How does function **return a value**?
 - rax** is used to **return a value**

Register	Purpose	Saved Across Calls
%rax	Temp register; Return value	No
%rbx	Callee-saved	Yes
%rcx	Used to pass 4th argument	No
%rdx	Used to pass 3rd argument	No
%rsp	Stack pointer	Yes
%rbp	Callee-saved; Base pointer	Yes
%rsi	Used to pass 2nd argument	No
%rdi	Used to pass 1st argument	No
%r8	Used to pass 5th argument	No
%r9	Used to pass 6th argument	No
%r10-r11	Temporary	No
%r12-r15	Callee-saved registers	Yes

```
int test(int a, int b, int c, int d) {  
    printf("%d %d %d %d\n", a, b, c, d);  
    return 3;  
}  
  
int main(void) {  
    int a = 0;  
    int b = 1;  
    int c = 2;  
    int d = 3;  
  
    test(a, b, c, d);  
    return 0;  
}
```

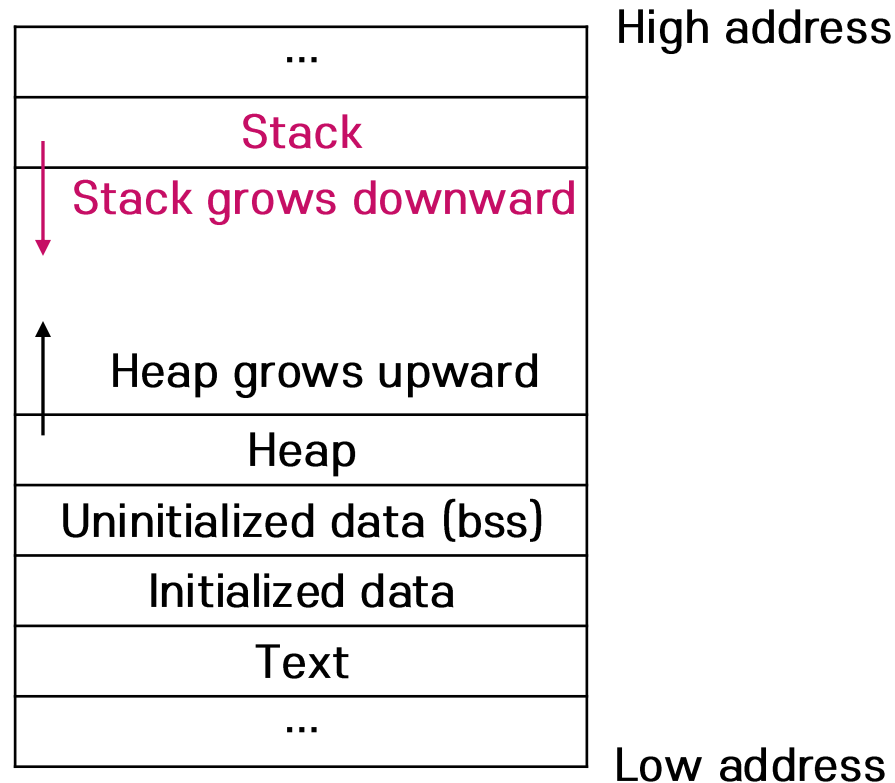
```
0x0000000000401155 <+47>: call 0x401000 <printf@plt>  
0x000000000040115a <+52>: mov $0x3,%eax
```

Today's Agenda

- GCC
- GDB
- Assembly Language
- **Memory Layout**
- Bomb Lab
- Quiz

Memory Layout: Stack

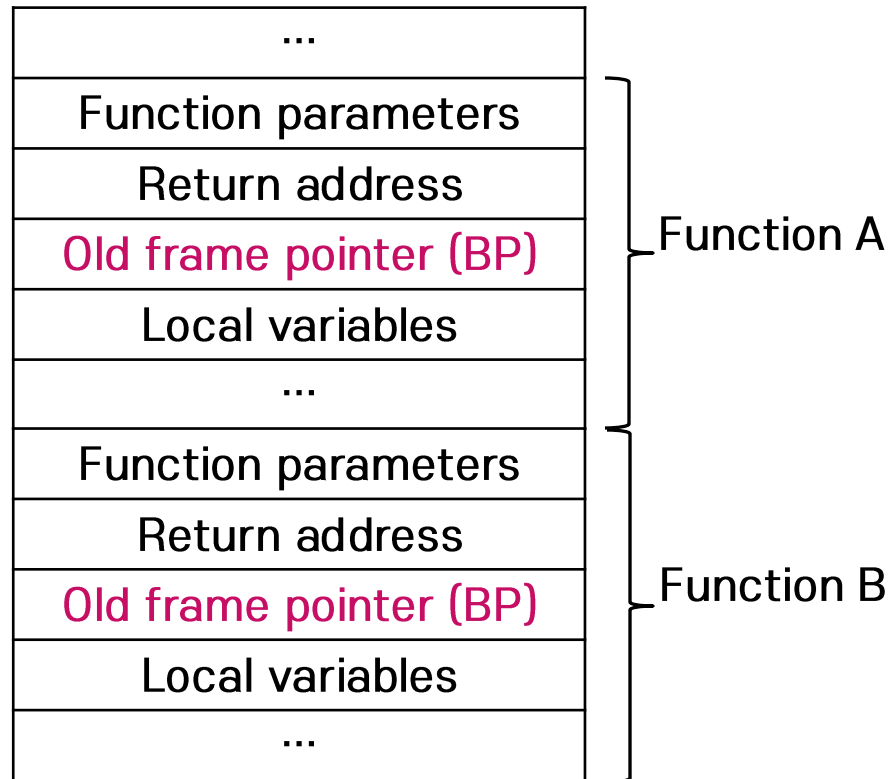
- Each program (binary) has its own address space
 - Stack is a LIFO (Last-In-First-out) data structure
 - Local variables are stored in stack
 - Stack can grow (downward) and shrink (upward) as a program is being executed



Memory Layout: Stack

- Stack frame
 - Stack stores information about subroutines (e.g., functions)
 - Stack is managed by
 - Pushing (prologue) and popping (epilogue) frame pointers (e.g., `rbp`, `rsp`)

High address



Low address

Function prologue:

- `push %rbp`
- `mov %rsp, %rbp`
- `sub N, %rsp`

Function epilogue:

- `mov %rbp, %rsp`
- `pop %rbp`
- `ret`

Today's Agenda

- GCC
- GDB
- Assembly Language
- Memory Layout
- **Bomb Lab**
- Quiz

Assignment: Bomb Lab

- Oh no!! Dr. Evil has written an evil program that will “explode” your programming server machines!
- The program is in 6 phases, each of which reads in input – something like a password – from standard input
- If your input is correct, you go on to the next phases
- If not, the bomb explodes. The program prints “BOOM!!!” and terminates, and you loss half a point (Your score is updated automatically)

Assignment: Bomb Lab

- Your goal is to defuse bomb in the given assembly file consists of 6 phases
 - If the bomb explode, you will get score penalty (0.5 pt.)
- Due: 10/15 (Wed) 23:59 (midnight)
 - Late submission will not be accepted
- Submit only your lab report (in pdf)
 - Report name: [student id].pdf (e.g., 20241234.pdf)
 - Briefly explain your progress in defusing the bomb. You must include:
 - Your bomb input solution
 - Your report should be less than 10 pages with font Arial and font size 11pt
- Evaluation: Quiz (10 pt.) + Report (90 pt.)
 - The weight of each phase is for [10, 15, 15, 20, 20, 20]%
 - No point, If there is no explicit explanation of how you solve each phase

Announcement

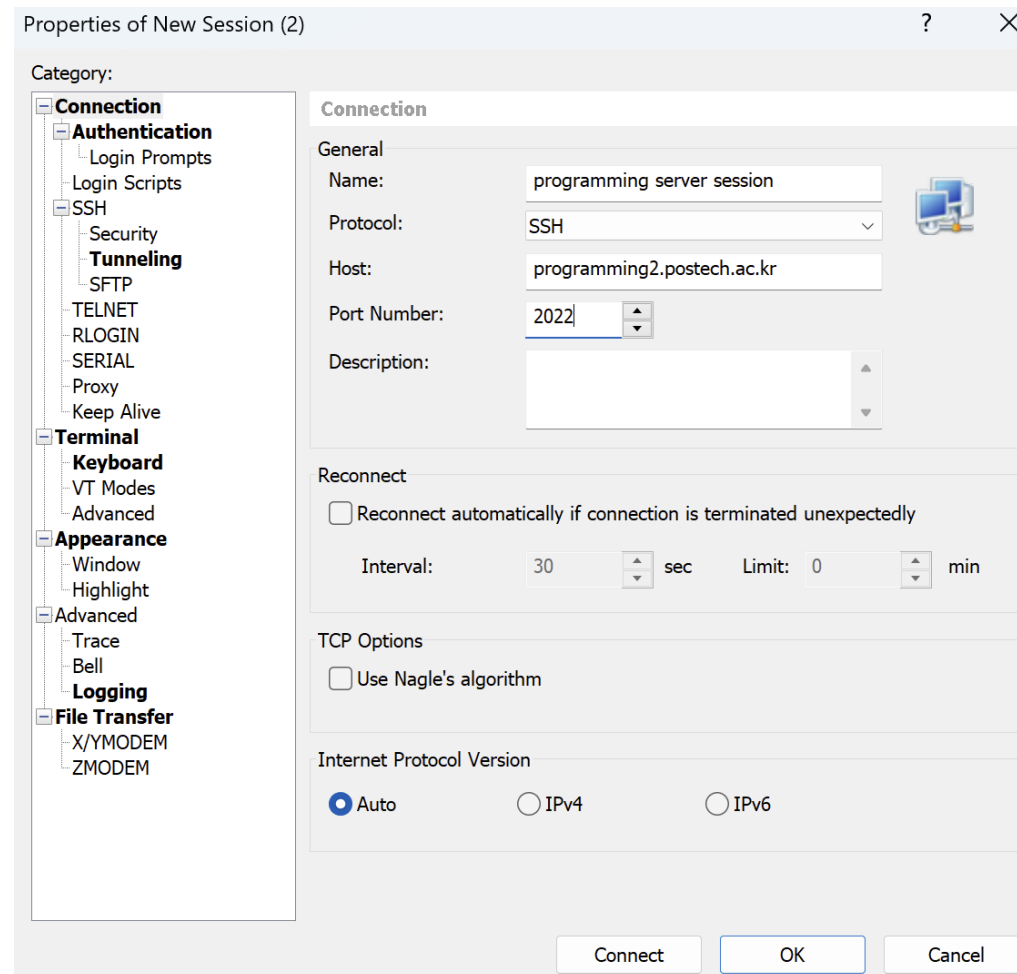
- If you submit your information, the bomb will be downloaded immediately
 - Bombs will **be created randomly** based on the order
 - You must **download bomb only once**
 - If you download it **more than once**, complete the assignment using the **initially downloaded bomb**
 - Your score will be based on the **initially listed on the scoreboard**
- The bomb server **will be opened** at 9/29 (Mon) 13:00 **after TA session**
 - If there is **any error** in bomb server, **send email to TA**

How to Download Bomb? (Terminal Version)

- Connect to the **programming server** with **ssh** command
 - `$ ssh -p 2022 -L 15213:127.0.0.1:15213`
`[YourServerID]@programming2.postech.ac.kr`
 - `-p`: **SSH port number** to connect server
 - `-L`: **Port tunneling**
- You can access the web server **only when the SSH session is alive**

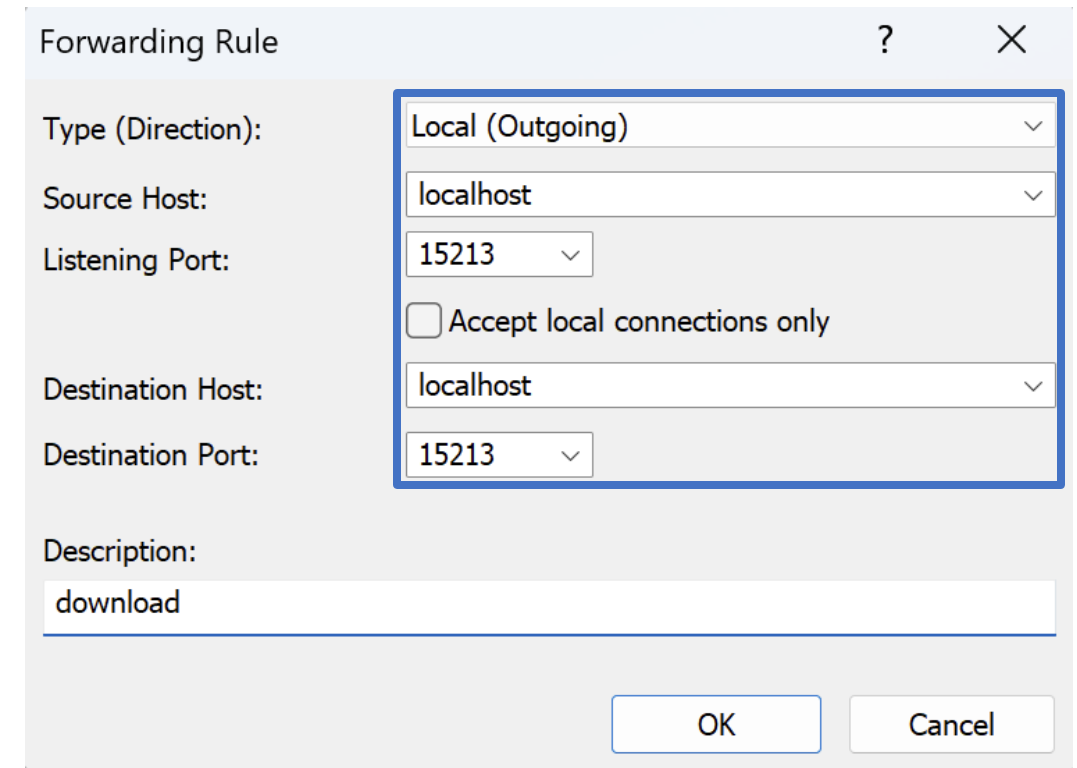
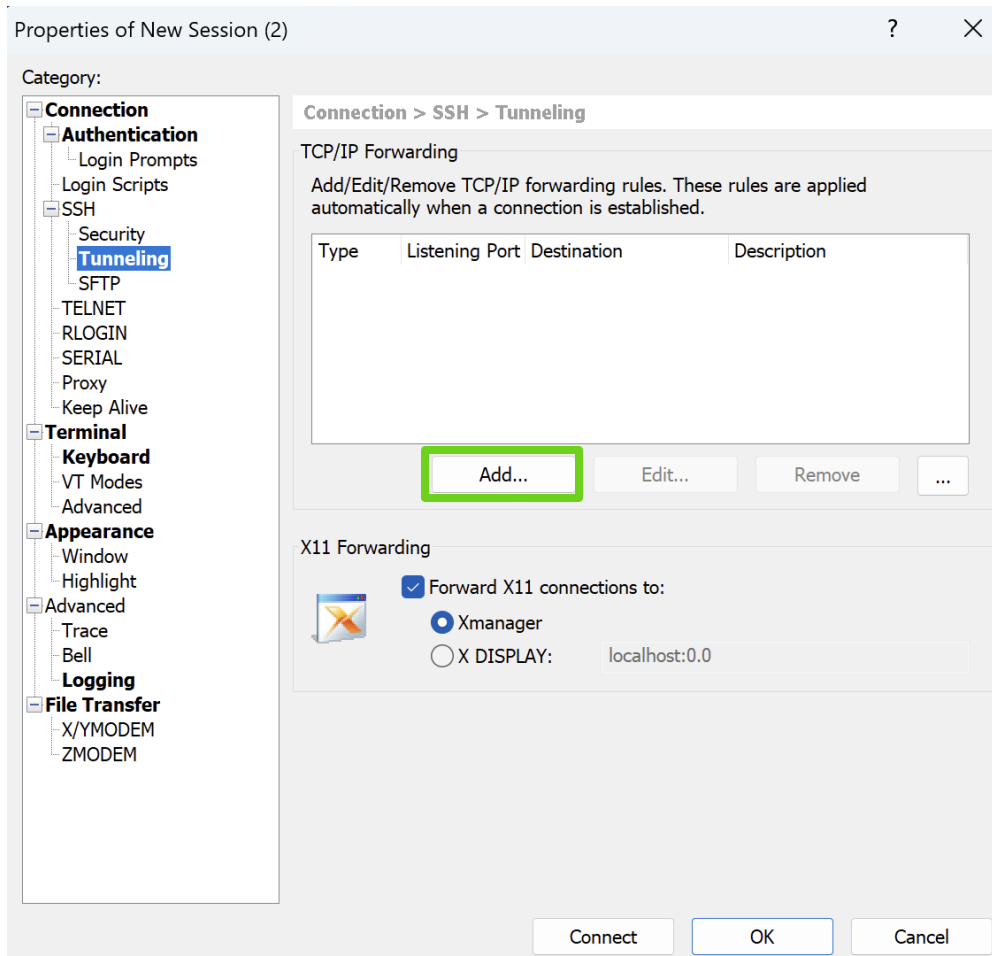
How to Download Bomb? (Xshell Version)

- Create new session

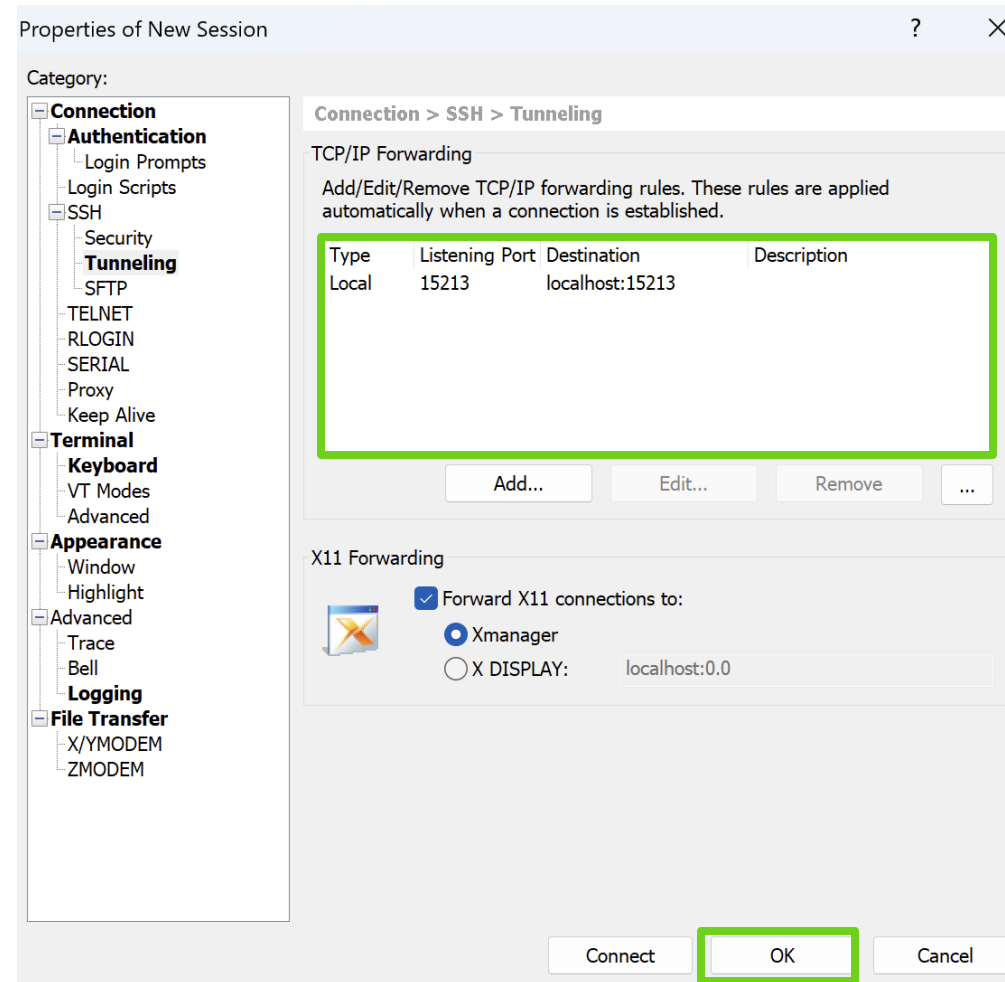


How to Download Bomb? (Xshell Version)

- Go to SSH → Tunneling to setup port forwarding
 - Add the forwarding rule for **15213**

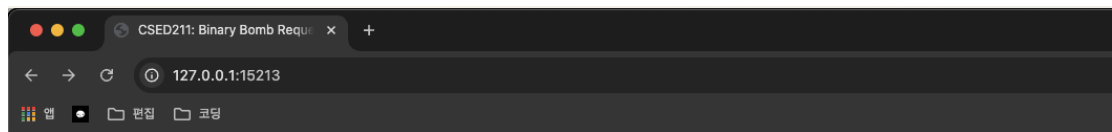


How to Download Bomb? (Xshell Version)



Accessing the Bomb Server

- In your web browser (e.g., Chrome), search 127.0.0.1:15213
- You can access the web server **only when the SSH session is alive**



CSED211: Binary Bomb Request

Fill in the form and then click the Submit button.

If you enter submit, bomb will be immediately downloaded to your local.

Hit the Reset button to get a clean form.

Legal characters are spaces, letters, numbers, underscores ('_'),
hyphens ('-'), at signs ('@'), and dots ('.').

Student ID

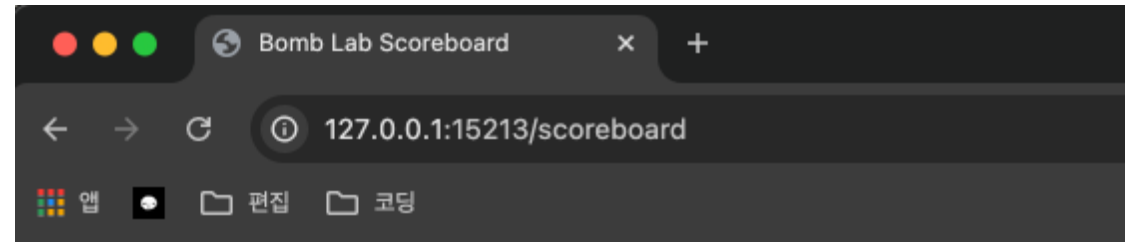
Enter your Student ID

Email address

If you submit your information,
the bomb will be downloaded immediately

Accessing the Bomb Scoreboard

- In your web browser (e.g., Chrome), **search 127.0.0.1:15213/scoreboard**
- You can access the web server **only when the SSH session is alive**



Bomb Lab Scoreboard

This page contains the latest information that we have received from your bomb. If your solution is marked **invalid**, this means your bomb reported a solution that didn't actually defuse your bomb.

Last updated: Thu Sep 19 23:28:51 2024 (updated every 30 secs)

#	Bomb number	Submission date	Phases defused	Explosions	Score	Status
---	-------------	-----------------	----------------	------------	-------	--------

Summary [phase:cnt] [1:0] total defused = 0/0

Practice

- Write the C program
 - `$ vi test.c`
- Compile the source code
 - `$ gcc -g -o test test.c`
- Run GDB over the program
 - `$ gdb test`

```
#include <stdio.h>

void practice_function(int x, int y);

int main(void) {
    int x, y;
    printf("Which one is larger? ");
    scanf("%d %d", &x, &y);
    practice_function(x, y);

    return 0;
}

void practice_function(int x, int y) {
    if (x > y) {
        printf("First one is larger\n");
    } else if (x < y) {
        printf("Second one is larger\n");
    } else {
        printf("Both are same\n");
    }
}
```

Practice (Cont'd)

- Disassemble the `main` function
 - Which are **called** in the `main` function?
 - How are **arguments passed** to functions?

```
0x000000000401146 <+0>:  push  %rbp
0x000000000401147 <+1>:  mov   %rsp,%rbp
0x00000000040114a <+4>:  sub   $0x10,%rsp
0x00000000040114e <+8>:  mov   $0x402004,%edi
0x000000000401153 <+13>: mov   $0x0,%eax
0x000000000401158 <+18>: call  0x401040 <printf@plt>
0x00000000040115d <+23>: lea    -0x8(%rbp),%rdx
0x000000000401161 <+27>: lea    -0x4(%rbp),%rax
0x000000000401165 <+31>: mov   %rax,%rsi
0x000000000401168 <+34>: mov   $0x40201a,%edi
0x00000000040116d <+39>: mov   $0x0,%eax
0x000000000401172 <+44>: call  0x401050 <__isoc99_scanf@plt>
0x000000000401177 <+49>: mov   -0x8(%rbp),%edx
0x00000000040117a <+52>: mov   -0x4(%rbp),%eax
0x00000000040117d <+55>: mov   %edx,%esi
0x00000000040117f <+57>: mov   %eax,%edi
0x000000000401181 <+59>: call  0x40118d <practice_function>
0x000000000401186 <+64>: mov   $0x0,%eax
0x00000000040118b <+69>: leave
0x00000000040118c <+70>: ret
```

```
int main(void) {
    int x, y;
    printf("Which one is larger? ");
    scanf("%d %d", &x, &y);
    practice_function(x, y);

    return 0;
}
```


Practice (Cont'd)

- Inspect arguments of `printf`
 - `printf` takes the **first argument as string**
 - The first argument is stored in `%edi`

```
int main(void) {  
    int x, y;  
    printf("Which one is larger? ");  
    scanf("%d %d", &x, &y);  
    practice_function(x, y);  
  
    return 0;  
}
```

End of assembler dump.

(gdb) x/s 0x402004

0x402004: "Which one is larger? "

(gdb) x/s 0x40201a

0x40201a: "%d %d"

```
0x0000000000401146 <+0>: push    %rbp  
0x0000000000401147 <+1>: mov     %rsp, %rbp  
0x000000000040114a <+4>: sub     $0x17, %rsp  
0x000000000040114e <+8>: mov     $0x402004, %edi  
0x0000000000401153 <+13>: mov     $0x0, %eax  
0x0000000000401158 <+18>: call    0x401040 <printf@plt>  
0x000000000040115d <+23>: lea     -0x8(%rbp), %rdx  
0x0000000000401161 <+27>: lea     -0x4(%rbp), %rax  
0x0000000000401165 <+31>: mov     %rax, %rsi  
0x0000000000401168 <+34>: mov     $0x40201a, %edi  
0x000000000040116d <+39>: mov     $0x0, %eax  
0x0000000000401172 <+44>: call    0x401050 <__isoc99_scanf@plt>  
0x0000000000401177 <+49>: mov     -0x8(%rbp), %edx  
0x000000000040117a <+52>: mov     -0x4(%rbp), %eax  
0x000000000040117d <+55>: mov     %edx, %esi  
0x000000000040117f <+57>: mov     %eax, %edi  
0x0000000000401181 <+59>: call    0x40118d <practice_function>  
0x0000000000401186 <+64>: mov     $0x0, %eax  
0x000000000040118b <+69>: leave  
0x000000000040118c <+70>: ret
```

Practice (Cont'd)

- Inspect arguments of `scanf`
 - `scanf` takes the **first argument as string**
 - The first argument is stored in `%edi`

```
int main(void) {  
    int x, y;  
    printf("Which one is larger? ");  
    scanf("%d %d", &x, &y);  
    practice_function(x, y);  
  
    return 0;  
}
```

End of assembler dump.

(gdb) x/s 0x402004

0x402004: "Which one is larger? "

(gdb) x/s 0x40201a

0x40201a: "%d %d"

```
0x0000000000401146 <+0>: push    %rbp  
0x0000000000401147 <+1>: mov     %rsp,%rbp  
0x000000000040114a <+4>: sub     $0x10,%rsp  
0x000000000040114e <+8>: mov     $0x402004,%edi  
0x0000000000401153 <+13>: mov     $0x0,%eax  
0x0000000000401158 <+18>: call    0x401040 <printf@plt>  
0x000000000040115d <+23>: lea     -0x8(%rbp),%rdx  
0x0000000000401161 <+27>: lea     -0x4(%rbp),%rax  
0x0000000000401165 <+31>: mov     %rax,%rsi  
0x0000000000401168 <+34>: mov     $0x40201a,%edi  
0x000000000040116d <+39>: mov     $0x0,%eax  
0x0000000000401172 <+44>: call    0x401050 <__isoc99_scanf@plt>  
0x0000000000401177 <+49>: mov     -0x8(%rbp),%edx  
0x000000000040117a <+52>: mov     -0x4(%rbp),%eax  
0x000000000040117d <+55>: mov     %edx,%esi  
0x000000000040117f <+57>: mov     %eax,%edi  
0x0000000000401181 <+59>: call    0x40118d <practice_function>  
0x0000000000401186 <+64>: mov     $0x0,%eax  
0x000000000040118b <+69>: leave  
0x000000000040118c <+70>: ret
```


Practice (Cont'd)

- Inspect arguments of `scanf`
 - 2nd and 3rd arguments are passed to `rsi` and `rdi`, respectively
 - Why `lea` instructions are used?
 - Because `&x` and `&y` are pointers (e.g., pointing to addresses of `x` and `y`)

```
int main(void) {  
    int x, y;  
    printf("Which one is larger? ");  
    scanf("%d %d", &x, &y);  
    practice_function(x, y);  
  
    return 0;  
}
```

```
0x0000000000401146 <+0>:    push    %rbp  
0x0000000000401147 <+1>:    mov     %rsp,%rbp  
0x000000000040114a <+4>:    sub     $0x10,%rsp  
0x000000000040114e <+8>:    mov     $0x402004,%edi  
0x0000000000401153 <+13>:   mov     $0x0,%eax  
0x0000000000401158 <+18>:   call    0x401040 <printf@plt>  
0x000000000040115d <+23>:   lea     -0x8(%rbp),%rdx  
0x0000000000401161 <+27>:   lea     -0x4(%rbp),%rax  
0x0000000000401165 <+31>:   mov     %rax,%rsi  
0x0000000000401168 <+34>:   mov     $0x402004,%edi  
0x000000000040116d <+39>:   mov     $0x0,%eax  
0x0000000000401172 <+44>:   call    0x401050 <__isoc99_scanf@plt>  
0x0000000000401177 <+49>:   mov     -0x8(%rbp),%edx  
0x000000000040117a <+52>:   mov     -0x4(%rbp),%eax  
0x000000000040117d <+55>:   mov     %edx,%esi  
0x000000000040117f <+57>:   mov     %eax,%edi  
0x0000000000401181 <+59>:   call    0x40118d <practice_function>  
0x0000000000401186 <+64>:   mov     $0x0,%eax  
0x000000000040118b <+69>:   leave  
0x000000000040118c <+70>:   ret
```

Practice (Cont'd)

- Inspect the register values before a point (e.g., `practice_function`)
 - Make a **breakpoint** at `practice_function`
 - `(gdb) b *practice_function`
 - **Run** the program
 - `(gdb) r`
 - Provide the program with **two integer arguments**

```
(gdb) b *practice_function
Breakpoint 1 at 0x40118d: file prac.c, line 14.
(gdb) r
Starting program: /root/prac
warning: Error disabling address space randomization: Operation not permitted
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Which one is larger? 5 6

Breakpoint 1, practice_function (x=32590, y=-1765326112) at prac.c:14
14      void practice_function(int x, int y) {
```

Practice (Cont'd)

- Inspect branch statements
 - x**: `%edi` $\rightarrow$ `-0x4(%rbp)`
 - y**: `%esi` $\rightarrow$ `-0x8(%rbp)`

```
void practice_function(int x, int y) {  
    if (x > y) {  
        printf("First one is larger\n");  
    } else if (x < y) {  
        printf("Second one is larger\n");  
    } else {  
        printf("Both are same\n");  
    }  
}
```

```
0x000000000040118d <+0>:  push    %rbp  
0x000000000040118e <+1>:  mov     %rsp,%rbp  
0x0000000000401191 <+4>:  sub     $0x10,%rsp  
0x0000000000401195 <+8>:  mov     %edi,-0x4(%rbp)  
0x0000000000401198 <+11>:  mov     %esi,-0x8(%rbp)  
0x000000000040119b <+14>:  mov     -0x4(%rbp),%eax  
0x000000000040119e <+17>:  cmp     -0x8(%rbp),%eax  
0x00000000004011a1 <+20>:  jle     0x4011af <practice_function+34>  
0x00000000004011a3 <+22>:  mov     $0x402020,%edi  
0x00000000004011a8 <+27>:  call    0x401030 <puts@plt>  
0x00000000004011ad <+32>:  jmp     0x4011cd <practice_function+64>  
0x00000000004011af <+34>:  mov     -0x4(%rbp),%eax  
0x00000000004011b2 <+37>:  cmp     -0x8(%rbp),%eax  
0x00000000004011b5 <+40>:  jge     0x4011c3 <practice_function+54>  
0x00000000004011b7 <+42>:  mov     $0x402034,%edi  
0x00000000004011bc <+47>:  call    0x401030 <puts@plt>  
0x00000000004011c1 <+52>:  jmp     0x4011cd <practice_function+64>  
0x00000000004011c3 <+54>:  mov     $0x402049,%edi  
0x00000000004011c8 <+59>:  call    0x4010
```

Questions?

Today's Agenda

- GCC
- GDB
- Assembly Language
- Bomb Lab
- **Quiz**

Quiz

- Q1. Write the assembly `opcode` that stores an address
- Q2. Write the GDB command to view the main function in assembly
- Go to PLMS, and start the quiz!